

整体架构

项目是一个“智能机器人巡检监测平台”，采用前后端分离架构：

前端：Vue 单页应用

后端：Django REST API

边缘端：Python 机器人侧程序，负责视频读取、AI 识别、遥测上报

视频服务：ZLMediaKit

本地数据库：PostgreSQL

部署辅助：Docker Compose

前端技术栈

配置来源：package.json

Vue 3：前端框架

Vue Router 4：页面路由

Vite 4：前端构建与开发服务器

ECharts 6：图表展示

vue-echarts：Vue 中集成 ECharts

mpegts.js：HTTP-FLV 视频播放

hls.js：HLS 视频播放

后端技术栈

配置来源：requirements.txt、settings.py

Python

Django 6.0.4：后端 Web 框架

Django REST Framework 3.17.1：REST API

django-cors-headers 4.9.0：跨域支持

DRF Token Authentication：接口 Token 鉴权

SQLite: 本地开发数据库

Django Media 文件系统: 保存抓拍图片和媒体文件

机器狗边缘端技术栈

配置来源: pyproject.toml

Python 3.10+

FastAPI: 边缘端 Demo 服务

Uvicorn: ASGI 服务运行器

OpenCV: 视频帧读取与图像处理

Ultralytics YOLO: 自行车/违停视觉识别

PyYAML: 读取配置文件

Requests: 向 Django 后端上报遥测、事件和媒体

ffmpeg: RTSP 拉流并推送 RTMP

本地 YAML 配置: 管理视频源、机器人编号、推流地址等

视频流技术栈

配置来源: docker-compose.zlmediakit.yml、robot-video-ai-solution.md

ZLMediaKit: 流媒体服务

RTSP: 机器人摄像头原始视频源

RTMP: 边缘端向 ZLMediaKit 推流

HTTP-FLV: 浏览器端优先播放格式

HLS: 浏览器端备用播放格式

WebRTC: 端口已预留, 后续可用于低延迟播放

机器人 RTSP 视频源 -> ffmpeg 推 RTMP -> ZLMediaKit 转 HTTP-FLV / HLS -> Vue 前端播放

开发与运行工具

npm: 前端依赖管理

Vite dev server: 前端开发服务

Django manage.py: 后端迁移与启动

Docker Compose: 启动 ZLMediaKit

PostgreSQL数据库: 数据存储

ffmpeg: 视频流转推